

▼ 데이터 확인하기

0. 라이브러리 импорт & 경로 설정

```
# ——— 환경 자동 감지 ———
try:
    import google.colab
    ENV = "colab"
except ImportError:
    ENV = "local"

if ENV == "local":
    BASE_DIR = "./dataset"
    RUNS_DIR = "runs/detect"
    FONT_PATH = "C:/Windows/Fonts/malgun.ttf" # 윈도우 맑은 고딕
    # FONT_PATH = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf" # 리눅스/Mac

elif ENV == "colab":
    from google.colab import drive
    drive.mount('/content/drive')
    BASE_DIR = "/content/drive/MyDrive/data/초급_프로젝트/dataset"
    RUNS_DIR = "/content/drive/MyDrive/data/초급_프로젝트/runs/detect"

    # Colab 한글 폰트 설치
    import subprocess
    subprocess.run(["apt-get", "install", "-y", "fonts-nanum"], capture_output=True)
    FONT_PATH = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf"

# ——— 한글 폰트 적용 ———
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm

fm.fontManager.addfont(FONT_PATH)
font_name = fm.FontProperties(fname=FONT_PATH).get_name()
plt.rcParams["font.family"] = font_name
plt.rcParams["axes.unicode_minus"] = False

print(f" 환경: {ENV}")
print(f" 폰트: {font_name}")
print(f" BASE_DIR: {BASE_DIR}")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
환경: colab
폰트: NanumGothic
BASE_DIR: /content/drive/MyDrive/data/초급_프로젝트/dataset
```

0. 라이브러리 импорт & 경로 설정

```
import os
import json
import pandas as pd
from PIL import Image
import unicodedata # 0번 섹션에 추가 필요
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader
import torch.optim as optim
import torchvision
import torchvision.transforms as T
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
import matplotlib
import matplotlib.pyplot as plt
from collections import Counter
import subprocess
import matplotlib as mpl
import numpy as np
import matplotlib.patches as patches
!pip3 install ultralytics -q
from ultralytics import YOLO
from ultralytics.utils import LOGGER
import logging
import shutil
import unicodedata
!pip3 install iterative-stratification -q
from iterstrat.ml_stratifiers import MultilabelStratifiedShuffleSplit
```

Creating new Ultralytics Settings v0.0.6 file  1.2/1.2 MB 82.0 MB/s eta 0:00:00
 View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
 Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com/quickstart/#ultralytics-settings>

```
import unicodedata

# ——— 디바이스 ———
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {DEVICE}")

# # ——— 한글 폰트 ———
# subprocess.run(['apt-get', 'install', '-y', 'fonts-nanum'], capture_output=True)
# subprocess.run(['fc-cache', '-fv'], capture_output=True)
# mpl.font_manager.FontManager.addfont('/usr/share/fonts/truetype/nanum/NanumGothic.ttf')
# mpl.rc('font', family='NanumGothic')
# plt.rcParams['axes.unicode_minus'] = False

# ——— 경로 설정 ———
# BASE_DIR = unicodedata.normalize('NFC', "/content/drive/MyDrive/data/초급_프로젝트/dataset/")

TRAIN_JSON_PATH = os.path.join(BASE_DIR, "merged_annotations_train_final.json")
TEST_JSON_PATH = os.path.join(BASE_DIR, "merged_annotations_test_final.json")
TRAIN_IMG_DIR = os.path.join(BASE_DIR, "train_images")
TEST_IMG_DIR = os.path.join(BASE_DIR, "test_images")
PROCESSED_JSON_PATH = os.path.join(BASE_DIR, "processed_train_v3.json")
YOLO_DATASET_DIR = os.path.join(BASE_DIR, "yolo_dataset_v3")
YOLO_RUNS_DIR = os.path.join(BASE_DIR, "yolo_runs_v3")
SUBMISSION_PATH = os.path.join(BASE_DIR, "submission_v3.csv")

for d in [YOLO_DATASET_DIR, YOLO_RUNS_DIR]:
    os.makedirs(d, exist_ok=True)

# ——— 경로 검증 ———
for name, path in [
    ("TRAIN_JSON", TRAIN_JSON_PATH),
    ("TEST_JSON", TEST_JSON_PATH),
    ("TRAIN_IMG_DIR", TRAIN_IMG_DIR),
    ("TEST_IMG_DIR", TEST_IMG_DIR),
]:
    print(f" {name} : {'OK' if os.path.exists(path) else '없음'} ({path})")

Device: cuda
TRAIN_JSON : OK (/content/drive/MyDrive/data/초급_프로젝트/dataset/merged_annotations_train_final.json)
TEST_JSON : OK (/content/drive/MyDrive/data/초급_프로젝트/dataset/merged_annotations_test_final.json)
TRAIN_IMG_DIR : OK (/content/drive/MyDrive/data/초급_프로젝트/dataset/train_images)
TEST_IMG_DIR : OK (/content/drive/MyDrive/data/초급_프로젝트/dataset/test_images)
```

✓ 1. 파일 존재 여부 및 기본 구조 확인

```
# 이미지 파일 개수 확인
IMG_EXTENSIONS = ('.jpg', '.jpeg', '.png', '.bmp')

def count_images(directory):
    """디렉토리 내 이미지 파일 수를 반환"""
    if not os.path.exists(directory):
        return 0
    return sum(
        1 for entry in os.scandir(directory)
        if entry.is_file() and entry.name.lower().endswith(IMG_EXTENSIONS)
    )

EXPECTED = {"Train": 1489, "Test": 843}

n_train = count_images(TRAIN_IMG_DIR)
n_test = count_images(TEST_IMG_DIR)

print(f"Train 이미지 수 : {n_train:,} (기대값: {EXPECTED['Train']:,})")
print(f"Test 이미지 수 : {n_test:,} (기대값: {EXPECTED['Test']:,})")
print(f"합계 : {n_train + n_test:,}")

if n_train != EXPECTED["Train"]:
    print(f"[경고] Train 이미지 수 불일치: {n_train} != {EXPECTED['Train']}")
if n_test != EXPECTED["Test"]:
    print(f"[경고] Test 이미지 수 불일치: {n_test} != {EXPECTED['Test']}")
```

```
Train 이미지 수 : 1,489 (기대값: 1,489)
Test 이미지 수 : 843 (기대값: 843)
합계 : 2,332
```

```
# 샘플 데이터 구조 확인
with open(TRAIN_JSON_PATH, 'r', encoding='utf-8') as f:
    train_json = json.load(f)

with open(TEST_JSON_PATH, 'r', encoding='utf-8') as f:
    test_json = json.load(f)

# --- 최상위 키 및 건수 ---
for label, data in [("Train", train_json), ("Test", test_json)]:
    print(f"[{label}] 최상위 키: {list(data.keys())}")
    print(f"   images      : {len(data.get('images', [])):.}건")
    print(f"   annotations : {len(data.get('annotations', [])):.}건")
    print(f"   categories  : {len(data.get('categories', [])):.}건")
    print()

# --- 샘플 1건씩 출력 ---
print("=== images[0] ===")
print(train_json['images'][0])

print("\n=== annotations[0] ===")
print(train_json['annotations'][0])

print("\n=== categories (앞 5개) ===")
for cat in train_json['categories'][:5]:
    print(cat)

# --- category_id 범위 및 연속성 확인 ---
cat_ids = sorted(c['id'] for c in train_json['categories'])
expected_ids = list(range(min(cat_ids), max(cat_ids) + 1))
is_continuous = cat_ids == expected_ids

print(f"\ncategory_id 범위 : {min(cat_ids)} ~ {max(cat_ids)}")
print(f"연속성      : {'연속' if is_continuous else '[경고] 불연속 (재매핑 필요)'}")
```

```
[Train] 최상위 키: ['images', 'annotations', 'categories']
images      : 1,489건
annotations : 4,526건
categories  : 73건

[Test] 최상위 키: ['images', 'annotations', 'categories']
images      : 843건
annotations : 1,129건
categories  : 73건

=== images[0] ===
{'file_name': 'K-001900-016551-024850-027926_0_2_0_2_90_000_200.png', 'width': 976, 'height': 1280, 'imgfile': 'K-001900-016551-024850-027926_0_2_0_2_90_000_200.png'}

=== annotations[0] ===
{'area': 25654, 'iscrowd': 0, 'bbox': [158, 251, 202, 127], 'category_id': 1899, 'ignore': 0, 'segmentation': [], 'id': 1, 'image_id': 1}

=== categories (앞 5개) ===
{'supercategory': 'pill', 'id': 1899, 'name': '보령부스파정 5mg'}
{'supercategory': 'pill', 'id': 27925, 'name': '울트라셋이알서방정'}
{'supercategory': 'pill', 'id': 16550, 'name': '동아가바펜틴정 800mg'}
{'supercategory': 'pill', 'id': 24849, 'name': '놀렉정 10mg'}
{'supercategory': 'pill', 'id': 29344, 'name': '비모보정 500/20mg'}

category_id 범위 : 1899 ~ 44198
연속성      : [경고] 불연속 (재매핑 필요)
```

3. 클래스 분포 분석

```
# 클래스 분포 확인
id2name = {cat['id']: cat['name'] for cat in train_json['categories']}
cat_counter = Counter(annot['category_id'] for annot in train_json['annotations'])

df_class = pd.DataFrame([
    {'category_id': cid, 'name': id2name.get(cid, '?'), 'count': cnt}
    for cid, cnt in cat_counter.most_common()
])

print(f"클래스 수 : {len(df_class)} | 전체 어노테이션 : {df_class['count'].sum():,}")
print(f"평균 : {df_class['count'].mean():.1f} | 중앙값 : {df_class['count'].median():.1f} | 최대/최소 비율 : {df_class['count'].max()/df_class['count'].min():.1f}")
print(f"\n최다 : {df_class.iloc[0]['name']} ({df_class.iloc[0]['count']}개)")
print(f"최소 : {df_class.iloc[-1]['name']} ({df_class.iloc[-1]['count']}개)")

print("\n최상위 5개:")
print(df_class.head(5)[['name', 'count']].to_string(index=False))
print("\n최하위 5개:")
print(df_class.tail(5)[['name', 'count']].to_string(index=False))
```

```
# 전체 분포 시각화
fig, ax = plt.subplots(figsize=(12, 4))
ax.bar(range(len(df_class)), df_class['count'], color='steelblue', edgecolor='none')
mean_val = df_class['count'].mean()
ax.axhline(mean_val, color='red', linestyle='--', linewidth=1)
ax.text(len(df_class) * 0.98, mean_val + 5, f"평균 {mean_val:.0f}개", color='red', fontsize=8, ha='right')
ax.set_title('클래스별 어노테이션 분포 (73개)', fontsize=11)
ax.set_xlabel('클래스 인덱스')
ax.set_ylabel('어노테이션 수')
plt.tight_layout()
plt.show()
```

클래스 수 : 73 | 전체 어노테이션 : 4,526
 평균 : 62.0 | 중앙값 : 35.0 | 최대/최소 비율 : 73.4x

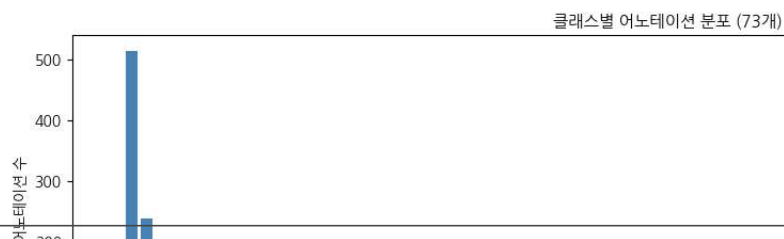
최다 : 기백신에프정(은행업엑스)(수출용) (514개)
 최소 : 브린텔릭스정 20mg (7개)

상위 5개:

	name	count
기백신에프정(은행업엑스)(수출용)		514
일양하이트린정 2mg	240	
보령부스파정 5mg	180	
유태란캡슐 100mg	172	
가바토파정 100mg	143	

하위 5개:

	name	count
줄로푸트정 100mg	11	
쿠에타핀정 25mg	10	
자이프렉사정 2.5mg	9	
구시노오정 10mg	8	
브린텔릭스정 20mg	7	



```
# 해상도 및 알약 수 분포 확인
for label, data in [("Train", train_json), ("Test", test_json)]:
    resolutions = Counter((img['width'], img['height']) for img in data['images'])
    print(f"[{label}] 해상도 분포:")
    for (w, h), cnt in resolutions.most_common():
        print(f" {w} x {h}: {cnt}장")

ann_per_image = Counter(ann['image_id'] for ann in train_json['annotations'])
no_ann = {img['id'] for img in train_json['images']} - set(ann_per_image.keys())
print(f"무annotation 없는 이미지: {len(no_ann)}장")

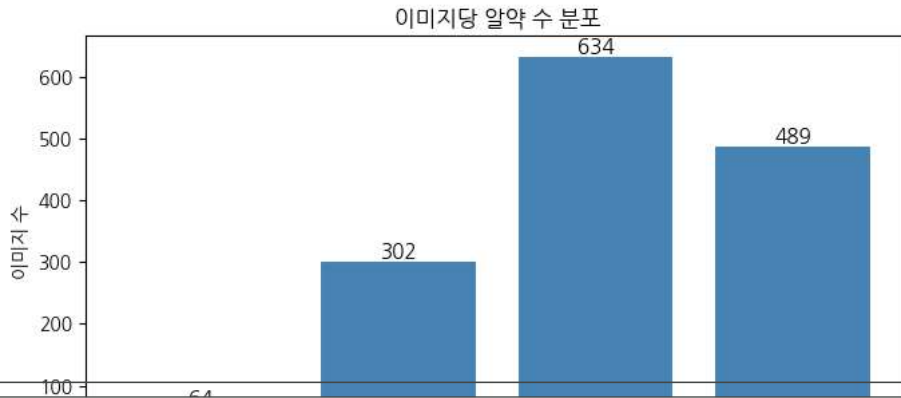
counts = list(ann_per_image.values())
pill_dist = Counter(counts)
print(f"평균 알약 수: {np.mean(counts):.2f}개 | 최대: {np.max(counts)}개")
for k in sorted(pill_dist.keys()):
    print(f" {k}개: {pill_dist[k]:4d}장 ({pill_dist[k]/len(train_json['images'])*100:.1f}%)")

fig, ax = plt.subplots(figsize=(7, 4))
keys, vals = zip(*sorted(pill_dist.items()))
bars = ax.bar(keys, vals, color='steelblue', edgecolor='white', linewidth=1.5)
for bar, v in zip(bars, vals):
    ax.text(bar.get_x() + bar.get_width()/2, v + 5, str(v), ha='center', fontsize=11, fontweight='bold')
ax.set_xlabel('이미지당 알약 수')
ax.set_ylabel('이미지 수')
ax.set_title('이미지당 알약 수 분포')
ax.set_xticks(keys)
plt.tight_layout()
plt.show()
```

[Train] 해상도 분포:
976 x 1280: 1489장
[Test] 해상도 분포:
976 x 1280: 843장

annotation 없는 이미지: 0장
평균 알약 수: 3.04개 | 최대: 4개

1개: 64장 (4.3%)
2개: 302장 (20.3%)
3개: 634장 (42.6%)
4개: 489장 (32.8%)



```
# 비정상 BBox 탐지
img_id2info = {img['id']: img for img in train_json['images']}
invalid_bboxes = []

for ann in train_json['annotations']:
    img = img_id2info[ann['image_id']]
    x, y, w, h = ann['bbox']
    iw, ih = img['width'], img['height']

    reasons = []
    if x < 0 or y < 0:
        reasons.append(f"음수 좌표 (x={x}, y={y})")
    if w <= 0 or h <= 0:
        reasons.append(f"비정상 크기 (w={w}, h={h})")
    if x + w > iw:
        reasons.append(f"x 범위 초과 (x+w={x+w} > {iw})")
    if y + h > ih:
        reasons.append(f"y 범위 초과 (y+h={y+h} > {ih})")
    if reasons:
        invalid_bboxes.append({
            'ann_id': ann['id'],
            'image_id': ann['image_id'],
            'bbox': ann['bbox'],
            'reason': ' / '.join(reasons)
        })

print(f"비정상 BBox 수: {len(invalid_bboxes)}개")
for item in invalid_bboxes:
    print(f"  ann_id={item['ann_id']} / image_id={item['image_id']} / bbox={item['bbox']} / 사유={item['reason']}")
```

비정상 BBox 수: 2개
ann_id=903 / image_id=239 / bbox=[6567, 625, 311, 315] / 사유=x 범위 초과 (x+w=6878 > 976)
ann_id=1157 / image_id=310 / bbox=[653, 8889, 217, 217] / 사유=y 범위 초과 (y+h=9106 > 1280)

```
# — file_name ↔ 실제 파일 매칭 검증 —————
actual_files = {entry.name for entry in os.scandir(TRAIN_IMG_DIR) if entry.is_file()}

missing = []
for img in train_json['images']:
    if img['file_name'] not in actual_files:
        missing.append(img['file_name'])

print(f"JSON 등록 이미지 수 : {len(train_json['images']):,}장")
print(f"실제 파일 수 : {len(actual_files):,}장")
print(f"매칭 실패 (누락 파일): {len(missing)}장")

if missing:
    print("\n누락 파일 목록:")
    for f in missing:
        print(f"  {f}")
else:
    print("모든 파일 정상 매칭")
```

JSON 등록 이미지 수 : 1,489장
실제 파일 수 : 1,489장
매칭 실패 (누락 파일): 0장
모든 파일 정상 매칭

5. 데이터 전처리

5-1. 비정상 BBox 제거

```
# 비정상 BBox 제거
invalid_ann_ids = {item['ann_id'] for item in invalid_bboxes}
clean_annotations = [ann for ann in train_json['annotations'] if ann['id'] not in invalid_ann_ids]

print(f"제거 전 : {len(train_json['annotations']):,}개")
print(f"제거 수 : {len(invalid_ann_ids)}개")
print(f"제거 후 : {len(clean_annotations):,}개")
```

제거 전 : 4,526개
제거 수 : 2개
제거 후 : 4,524개

5-2. category_id 재매핑

```
# category_id 재매핑 (원본 id → 0~72 연속 정수)
sorted_cats = sorted(train_json['categories'], key=lambda c: c['id'])

old_id2new = {cat['id']: new_id for new_id, cat in enumerate(sorted_cats)}
new_id2old = {new_id: cat['id'] for new_id, cat in enumerate(sorted_cats)}
new_id2name = {new_id: cat['name'].strip() for new_id, cat in enumerate(sorted_cats)}

remapped_annotations = []
for ann in clean_annotations:
    new_ann = ann.copy()
    new_ann['category_id'] = old_id2new[ann['category_id']]
    remapped_annotations.append(new_ann)

remapped_categories = [
    {'id': new_id, 'name': new_id2name[new_id], 'supercategory': 'pill'}
    for new_id in range(len(sorted_cats))
]

new_cat_ids = sorted(set(ann['category_id'] for ann in remapped_annotations))
print(f"category_id 범위 : {min(new_cat_ids)} ~ {max(new_cat_ids)}")
print(f"고유 클래스 수 : {len(new_cat_ids)}개")
print(f"'연속성' : {'연속' if new_cat_ids == list(range(len(new_cat_ids))) else '[경고] 불연속'}")
print(f"annotation 수 : {len(remapped_annotations):,}개")
```

category_id 범위 : 0 ~ 72
고유 클래스 수 : 73개
연속성 : 연속
annotation 수 : 4,524개

5-3. letterbox 리사이즈 (삭제)

976x1280

원본 사이즈 사용

5-4. train/val 분리

*Multi-label Stratified Split

동작 방식

- 이미지 내 모든 클래스를 동시에 고려해 분할
- 각 클래스의 train/val 비율이 동시에 80:20에 가깝도록 최적화
- iterative-stratification 라이브러리 사용

```
# Train/Val Multi-label Stratified Split
processed_annotations = remapped_annotations

# 이미지별 annotation 그룹화
img2anns = {}
for ann in processed_annotations:
    img2anns.setdefault(ann['image_id'], []).append(ann)

img_ids = np.array(sorted(img2anns.keys()))
n_classes = len(remapped_categories)

# 이미지별 멀티핫 벡터 생성 (73차원)
```

```

y = np.zeros((len(img_ids), n_classes), dtype=int)
for i, img_id in enumerate(img_ids):
    for ann in img2anns[img_id]:
        y[i, ann['category_id']] = 1

# 클래스당 이미지 1장인 경우 강제 train 배정
class_counts = y.sum(axis=0)
single_classes = np.where(class_counts == 1)[0]
force_train_mask = np.any(y[:, single_classes] == 1, axis=1) if len(single_classes) > 0 else np.zeros(len(img_ids), dtype=bool)

remaining_idx = np.where(~force_train_mask)[0]
force_idx = np.where(force_train_mask)[0]

# Multi-label stratified split
msss = MultilabelStratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, val_idx = next(msss.split(remaining_idx, y[remaining_idx]))

train_ids = img_ids[remaining_idx[train_idx]].tolist() + img_ids[force_idx].tolist()
val_ids = img_ids[remaining_idx[val_idx]].tolist()

# 검증
train_y = y[[i for i, img_id in enumerate(img_ids) if img_id in set(train_ids)]]
val_y = y[[i for i, img_id in enumerate(img_ids) if img_id in set(val_ids)]]

print(f"Train : {len(train_ids)}장 | Val : {len(val_ids)}장")
print(f"Train 클래스 수 : {train_y.sum(axis=0).astype(bool).sum()}개")
print(f"Val 클래스 수 : {val_y.sum(axis=0).astype(bool).sum()}개")
print(f"강제 train 배정 : {len(force_idx)}장")

```

```

Train : 1182장 | Val : 307장
Train 클래스 수 : 73개
Val 클래스 수 : 73개
강제 train 배정 : 0장

```

5-5. 전처리된 JSON 저장

```

# 전처리 JSON 저장
processed_json = {
    'images' : train_json['images'],
    'annotations': processed_annotations,
    'categories': remapped_categories,
    'train_ids' : list(train_ids),
    'val_ids' : list(val_ids),
    'mappings' : {
        'old_id2new': {str(k): v for k, v in old_id2new.items()},
        'new_id2old': {str(k): v for k, v in new_id2old.items()},
        'new_id2name': {str(k): v for k, v in new_id2name.items()},
    }
}

with open(PROCESSED_JSON_PATH, 'w', encoding='utf-8') as f:
    json.dump(processed_json, f, ensure_ascii=False, indent=2)

with open(PROCESSED_JSON_PATH, 'r', encoding='utf-8') as f:
    verify = json.load(f)

print(f"images : {len(verify['images']):,}장")
print(f"annotations : {len(verify['annotations']):,}개")
print(f"categories : {len(verify['categories']):,}개")
print(f"train_ids : {len(verify['train_ids']):,}장")
print(f"val_ids : {len(verify['val_ids']):,}장")
cat_ids = [ann['category_id'] for ann in verify['annotations']]
print(f"category_id : {min(cat_ids)} ~ {max(cat_ids)}")

```

```

images : 1,489장
annotations : 4,524개
categories : 73개
train_ids : 1,182장
val_ids : 307장
category_id : 0 ~ 72

```

5-6. 전처리 결과 검증

```

# 전처리 최종 검증
train_id_set = set(train_ids)
val_id_set = set(val_ids)
train_annotations = [ann for ann in processed_annotations if ann['image_id'] in train_id_set]
val_annotations = [ann for ann in processed_annotations if ann['image_id'] in val_id_set]

```

```
# 1. BBox 좌표 범위 검증
img_id2info = {img['id']: img for img in train_json['images']}
bbox_errors = []
for ann in processed_annotations:
    x, y, w, h = ann['bbox']
    iw = img_id2info[ann['image_id']]['width']
    ih = img_id2info[ann['image_id']]['height']
    reasons = []
    if x < 0 or y < 0: reasons.append(f"음수 좌표 (x={x:.1f}, y={y:.1f})")
    if w <= 0 or h <= 0: reasons.append(f"비정상 크기 (w={w:.1f}, h={h:.1f})")
    if x + w > iw: reasons.append(f"x 범위 초과 ({x+w:.1f} > {iw})")
    if y + h > ih: reasons.append(f"y 범위 초과 ({y+h:.1f} > {ih})")
    if reasons:
        bbox_errors.append({'ann_id': ann['id'], 'reason': ' / '.join(reasons)})

print(f"비정상 BBox : {len(bbox_errors)}개" + (f"\n {bbox_errors}" if bbox_errors else " (정상)"))

# 2. Train/Val 클래스 분포 비율 검증
total_cat_counts = Counter(ann['category_id'] for ann in processed_annotations)
train_cat_counts = Counter(ann['category_id'] for ann in train_annotations)
val_cat_counts = Counter(ann['category_id'] for ann in val_annotations)

print(f"\n{{'new_id':<8}} {{'클래스명':<30}} {{'전체':>6}} {{'Train':>6}} {{'Val':>5}} {{'Val비율':>8}}")
print("-" * 70)
ratio_errors = []
for cat_id in range(len(remapped_categories)):
    total = total_cat_counts.get(cat_id, 0)
    train = train_cat_counts.get(cat_id, 0)
    val = val_cat_counts.get(cat_id, 0)
    val_ratio = val / total if total > 0 else 0
    flag = " [경고]" if abs(val_ratio - 0.2) > 0.15 and total >= 5 else ""
    print(f" {{cat_id:<6}} {{new_id2name[cat_id]:<30}} {{total:>6}} {{train:>6}} {{val:>5}} {{val_ratio:>7.1%}}{flag}")
    if flag:
        ratio_errors.append(cat_id)

print(f"\n비율 이상 클래스 : {len(ratio_errors)}개")
print(f"Train 클래스 수 : {len(train_cat_counts)}개")
print(f"Val 클래스 수 : {len(val_cat_counts)}개")
```

비정상 BBox : 0개 (정상)

new_id	클래스명	전체	Train	Val	Val비율		
0	보령부스파정 5mg	180	144	36	20.0%		
1	뮤테란캡슐 100mg	172	140	32	18.6%		
2	일양하이트린정 2mg	240	192	48	20.0%		
3	기백신에프정(은행엽엑스)(수출용)		514	411	103	20.0%	
4	무코스타정(레바미피드)(비매품)		98	78	20	20.4%	
5	알드린정	54	43	11	20.4%		
6	뉴로메드정(옥시라세탐)		27	22	5	18.5%	
7	타이레놀정500mg	33	26	7	21.2%		
8	에어탈정(아세클로페낙)		29	23	6	20.7%	
9	삼남건조수산화알루미늄겔정		56	44	12	21.4%	
10	타이레놀이알서방정(아세트아미노펜)(수출용)			35	28	7	20.0%
11	베콤씨에프정 618.6mg/병	34	27	7	20.6%		
12	조인스정 200mg	35	28	7	20.0%		
13	세로켈정 100mg	35	28	7	20.0%		
14	리렉스펜정 300mg/PTP	35	28	7	20.0%		
15	아빌리파이정 10mg	16	13	3	18.8%		
16	자이프렉사정 2.5mg	9	7	2	22.2%		
17	다보타민큐정 10mg/병	36	29	7	19.4%		
18	써스펜8시간이알서방정 650mg		34	27	7	20.6%	
19	에빅사정(메탄티염산염)(비매품)		24	19	5	20.8%	
20	리피토정 20mg	109	87	22	20.2%		
21	크레스토정 20mg	108	86	22	20.4%		
22	가바토파정 100mg	143	114	29	20.3%		
23	동아가바펜틴정 800mg	139	111	28	20.1%		
24	오마코연질캡슐(오메가-3-산에틸에스테르90)			16	13	3	18.8%
25	란스톤엘에프디티정 30mg		24	19	5	20.8%	
26	리리카캡슐 150mg	23	18	5	21.7%		
27	종근당글리아티린연질캡슐(콜린알포세레이트)			21	17	4	19.0%
28	콜리네이트연질캡슐 400mg		26	21	5	19.2%	
29	트루비타정 60mg/병	39	31	8	20.5%		
30	스토가정 10mg	32	26	6	18.8%		
31	노바스크정 5mg	66	53	13	19.7%		
32	마도파정	23	18	5	21.7%		
33	플라빅스정 75mg	97	78	19	19.6%		
34	엑스포지정 5/160mg	80	64	16	20.0%		
35	펠루비정(펠루비프로펜)		35	28	7	20.0%	
36	아토르바정 10mg	26	21	5	19.2%		
37	라비에트정 20mg	29	23	6	20.7%		
38	리피로우정 20mg	26	21	5	19.2%		
39	자누비아정 50mg	85	68	17	20.0%		
40	맥시부펜이알정 300mg	36	29	7	19.4%		
41	메가파워정 90mg/병	36	29	7	19.4%		
42	쿠에타핀정 25mg	10	8	2	20.0%		

43	비타비백정 100mg/병	38	29	9	23.7%
44	놀텍정 10mg	27	22	5	18.5%
45	자누메트정 50/850mg	88	70	18	20.5%
46	큐시드정 31.5mg/PTP	61	47	14	23.0%
47	아모잘탄정 5/100mg	76	61	15	19.7%
48	세비카정 10/40mg	79	61	18	22.8%
49	트윈스티정 40/5mg	94	75	19	20.2%
50	카나브정 60mg	84	67	17	20.2%
51	울트라셋이알서방정	38	30	8	21.1%
52	솔로푸트정 100mg	11	9	2	18.2%

6. DataLoader 구성

모델 : YOLOv8m

현재 가장 널리 쓰이는 1-stage detector입니다. 속도와 정확도 균형이 뛰어나고, Ultralytics 라이브러리로 학습/추론 코드가 매우 간결합니다.

- 장점: 빠른 학습 속도, 코드 간결, 데이터 증강 내장, mAP 성능 우수
- 단점: YOLO 포맷으로 데이터 변환 필요
- 권장 batch_size: 8~16
- 권장 모델 크기: yolov8m (medium) - 속도/정확도 균형

모델

- 모델: YOLOv8m
- 사전학습 가중치: yolov8m.pt (COCO)

데이터

- 이미지 크기: 원본 해상도 (976×1280), rect=True
- 클래스 수: 73
- Train: 1,182장 / Val: 307장
- 포맷: YOLO (x_center, y_center, w, h 정규화)

학습 하이퍼파라미터

- epochs: 50
- batch_size: 16
- optimizer: AdamW
- learning_rate: 0.001
- image_size: 1280
- rect: True

데이터 증강 (YOLOv8 내장)

- Mosaic: 활성화
- MixUp: 활성화
- HSV 색상 변환: 비활성화
- degrees: 비활성화
- flipplr / flipud: 비활성화

학습 설정

- Early stopping: patience=20
- workers: 8
- 체크포인트 저장: Google Drive

미결정 사항 (추후 진행)

- 클래스 불균형 해소 전략 (Focal Loss / 클래스 가중치)
- confidence threshold
- IoU threshold (NMS)
- Kaggle 제출 포맷 변환 (YOLO 출력 → COCO 포맷)

6-1. 이미지 포맷 변환

COCO(기존 이미지 형식) [x, y, w, h] → YOLO [x_center, y_center, w, h]

심볼릭 링크(Symbolic Link)를 사용하여 실제 파일을 복사하지 않고, 원본 파일의 경로를 가리키는 포인터를 만들어 사용한다.

심볼릭 링크는 실제 파일 없이 원본을 참조하므로 용량 추가 없이 YOLO가 요구하는 디렉토리 구조를 만들 수 있습니다.

```

# YOLO 포맷 변환
YOLO_LABEL_TRAIN = os.path.join(YOLO_DATASET_DIR, "labels", "train")
YOLO_LABEL_VAL = os.path.join(YOLO_DATASET_DIR, "labels", "val")
YOLO_IMG_TRAIN = os.path.join(YOLO_DATASET_DIR, "images", "train")
YOLO_IMG_VAL = os.path.join(YOLO_DATASET_DIR, "images", "val")
YOLO_YAML_PATH = os.path.join(YOLO_DATASET_DIR, "data.yaml")

for d in [YOLO_LABEL_TRAIN, YOLO_LABEL_VAL, YOLO_IMG_TRAIN, YOLO_IMG_VAL]:
    os.makedirs(d, exist_ok=True)

def coco_to_yolo(bbox, img_w, img_h):
    x, y, w, h = bbox
    x_c = min(max((x + w / 2) / img_w, 0.0), 1.0)
    y_c = min(max((y + h / 2) / img_h, 0.0), 1.0)
    w_n = min(max(w / img_w, 0.0), 1.0)
    h_n = min(max(h / img_h, 0.0), 1.0)
    return x_c, y_c, w_n, h_n

img_id2anns = {}
for ann in processed_annotations:
    img_id2anns.setdefault(ann['image_id'], []).append(ann)

def write_yolo(img_ids, label_dir, img_dir):
    count = 0
    for img_info in train_json['images']:
        if img_info['id'] not in set(img_ids):
            continue
        label_path = os.path.join(label_dir, os.path.splitext(img_info['file_name'])[0] + '.txt')
        with open(label_path, 'w') as f:
            for ann in img_id2anns.get(img_info['id'], []):
                x_c, y_c, w_n, h_n = coco_to_yolo(ann['bbox'], img_info['width'], img_info['height'])
                f.write(f"{ann['category_id']} {x_c:.6f} {y_c:.6f} {w_n:.6f} {h_n:.6f}Wn")
        shutil.copy2(
            os.path.join(TRAIN_IMG_DIR, img_info['file_name']),
            os.path.join(img_dir, img_info['file_name'])
        )
        count += 1
    return count

n_train = write_yolo(train_ids, YOLO_LABEL_TRAIN, YOLO_IMG_TRAIN)
n_val = write_yolo(val_ids, YOLO_LABEL_VAL, YOLO_IMG_VAL)

yaml_content = f"""train: {YOLO_IMG_TRAIN}
val: {YOLO_IMG_VAL}

nc: {len(remapped_categories)}
names: {[new_id2name[i] for i in range(len(remapped_categories))]}
"""

with open(YOLO_YAML_PATH, 'w', encoding='utf-8') as f:
    f.write(yaml_content)

n_ti = len(os.listdir(YOLO_IMG_TRAIN))
n_vi = len(os.listdir(YOLO_IMG_VAL))
n_tl = len(os.listdir(YOLO_LABEL_TRAIN))
n_vl = len(os.listdir(YOLO_LABEL_VAL))

print(f"Train 이미지: {n_ti:.} / 라벨: {n_tl:.}")
print(f"Val 이미지: {n_vi:.} / 라벨: {n_vl:.}")
if n_ti != n_tl: print("[경고] Train 이미지/라벨 수 불일치")
if n_vi != n_vl: print("[경고] Val 이미지/라벨 수 불일치")

sample = os.listdir(YOLO_LABEL_TRAIN)[0]
print(f"Wn샘플 라벨 ({sample}):")
with open(os.path.join(YOLO_LABEL_TRAIN, sample)) as f:
    print(f.read())

```

Train 이미지: 1,182 / 라벨: 1,182
Val 이미지: 307 / 라벨: 307

샘플 라벨 (K-003351-018147-020238_0_2_0_2_90_000_200.txt):
2 0.482070 0.215625 0.187500 0.143750
33 0.482070 0.215625 0.187500 0.143750

```

import os
print("Train path exists:", os.path.exists(YOLO_IMG_TRAIN))
print("Val path exists:", os.path.exists(YOLO_IMG_VAL))

```

Train path exists: True
Val path exists: True

6-2. 모델 학습 코드

```
# YOLOv8m 학습
EXPERIMENT_NAME = "yolov8m_oral_drug_v3"

# 기록용 리스트 추가
train_history = []

def on_train_epoch_end(trainer):
    epoch = trainer.epoch + 1
    epochs = trainer.epochs
    box_loss = trainer.loss_items[0].item() if trainer.loss_items is not None else 0
    cls_loss = trainer.loss_items[1].item() if trainer.loss_items is not None else 0
    dfl_loss = trainer.loss_items[2].item() if trainer.loss_items is not None else 0
    print(f"[Epoch {epoch:3d}/{epochs}] box_loss: {box_loss:.4f} cls_loss: {cls_loss:.4f} dfl_loss: {dfl_loss:.4f}")

    # epoch별 loss 기록
    train_history.append({
        "epoch": epoch,
        "box_loss": box_loss,
        "cls_loss": cls_loss,
        "dfl_loss": dfl_loss,
    })

def on_val_end validator):
    metrics = validator.metrics
    map50 = metrics.box.map50 if hasattr(metrics, 'box') else 0
    map5095 = metrics.box.map if hasattr(metrics, 'box') else 0
    print(f"Val mAP50: {map50:.4f} mAP50-95: {map5095:.4f}")

    # mAP 기록 (마지막 epoch에 추가)
    if train_history:
        train_history[-1]["map50"] = map50
        train_history[-1]["map5095"] = map5095

def on_train_end(trainer):
    # 최고 mAP 기록 찾기
    best = max(train_history, key=lambda x: x.get("map5095", 0))
    print(f"Wn{' '*55}")
    print(f"학습 완료 | 저장 경로: {trainer.save_dir}")
    print(f"최고 성능 | Epoch {best['epoch']:3d} mAP50: {best.get('map50', 0):.4f} mAP50-95: {best.get('map5095', 0):.4f}")
    print(f"{' '*55}")

    # 학습 완료 후 DataFrame으로 저장
    df_history = pd.DataFrame(train_history)
    history_path = os.path.join(trainer.save_dir, "train_history.csv")
    df_history.to_csv(history_path, index=False)
    print(f"학습 기록 저장 : {history_path}")
```

```
model = YOLO("yolov8m.pt")
model.add_callback("on_train_epoch_end", on_train_epoch_end)
model.add_callback("on_val_end", on_val_end)
model.add_callback("on_train_end", on_train_end)
results = model.train(
    # 데이터 관련 설정
    data = YOLO_YAML_PATH, # 학습 데이터셋 YAML 파일 경로
    imgsz = 1280, # 입력 이미지 크기
    rect = True, # 직사각형 배치(rectangular training)

    # 학습 과정 제어
    epochs = 50, # 학습 반복 횟수
    batch = 16, # 배치 크기
    optimizer = "AdamW", # 최적화 알고리즘
    lr0 = 0.005, # 초기 학습률
    lrf = 0.01, # 최종 학습률 비율
    weight_decay = 0.0005, # 가중치 감소 (정규화)
    warmup_epochs = 3, # 워밍업 에포크 수
    patience = 15, # 조기 종료(Early Stopping) patience

    # 데이터 증강 옵션
    mosaic = 1.0, # Mosaic 증강 비율
    mixup = 0.1, # MixUp 증강 비율
    hsv_h = 0.0, # HSV 색상(Hue) 변환 범위
    hsv_s = 0.0, # HSV 채도(Saturation) 변환 범위
    hsv_v = 0.0, # HSV 명도(Value) 변환 범위
    degrees = 0.0, # 회전 각도 범위
    flip_lr = 0.0, # 좌우 반전 확률
    flipud = 0.0, # 상하 반전 확률
```

```
# 결과 저장 및 관리
save      = True,          # 학습 결과 저장 여부
project   = YOLO_RUNS_DIR, # 결과 저장할 프로젝트 디렉토리
name      = EXPERIMENT_NAME, # 실험 이름
exist_ok  = True,          # 동일 이름 폴더 덮어쓰기 허용

# 환경 설정
device    = 0,             # 학습에 사용할 디바이스 (GPU 0번)
workers   = 8,             # 데이터 로딩 워커 수
verbose   = False,         # 학습 로그 출력 여부

)
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8m.pt> to 'yolov8m.pt': 100% ————— 4
 Ultralytics 8.4.31 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=16, bgr=0.0, box=7.5, cache=
 Downloading <https://ultralytics.com/assets/Arial.Unicode.ttf> to '/root/.config/Ultralytics/Arial.Unicode.ttf': 100% —————
 Overriding model.yaml nc=80 with nc=73

	from	n	params	module	arguments	
0		-1	1	1392	ultralytics.nn.modules.conv.Conv	[3, 48, 3, 2]
1		-1	1	41664	ultralytics.nn.modules.conv.Conv	[48, 96, 3, 2]
2		-1	2	111360	ultralytics.nn.modules.block.C2f	[96, 96, 2, True]
3		-1	1	166272	ultralytics.nn.modules.conv.Conv	[96, 192, 3, 2]
4		-1	4	813312	ultralytics.nn.modules.block.C2f	[192, 192, 4, True]
5		-1	1	664320	ultralytics.nn.modules.conv.Conv	[192, 384, 3, 2]
6		-1	4	3248640	ultralytics.nn.modules.block.C2f	[384, 384, 4, True]
7		-1	1	1991808	ultralytics.nn.modules.conv.Conv	[384, 576, 3, 2]
8		-1	2	3985920	ultralytics.nn.modules.block.C2f	[576, 576, 2, True]
9		-1	1	831168	ultralytics.nn.modules.block.SPPF	[576, 576, 5]
10		-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11		[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
12		-1	2	1993728	ultralytics.nn.modules.block.C2f	[960, 384, 2]
13		-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14		[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
15		-1	2	517632	ultralytics.nn.modules.block.C2f	[576, 192, 2]
16		-1	1	332160	ultralytics.nn.modules.conv.Conv	[192, 192, 3, 2]
17		[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat	[1]
18		-1	2	1846272	ultralytics.nn.modules.block.C2f	[576, 384, 2]
19		-1	1	1327872	ultralytics.nn.modules.conv.Conv	[384, 384, 3, 2]
20		[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat	[1]
21		-1	2	4207104	ultralytics.nn.modules.block.C2f	[960, 576, 2]
22		[15, 18, 21]	1	3817963	ultralytics.nn.modules.head.Detect	[73, 16, None, [192, 384, 576]]

Model summary: 170 layers, 25,898,587 parameters, 25,898,571 gradients, 79.3 GFLOPs

Transferred 469/475 items from pretrained weights

Freezing layer 'model.22.dfl.conv.weight'

AMP: running Automatic Mixed Precision (AMP) checks...

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt> to 'yolo26n.pt': 100% ————— 5

AMP: checks passed ☒

train: Fast image access ☒ (ping: 0.3±0.1 ms, read: 565.8±114.4 MB/s, size: 1738.2 KB)

train: Scanning /content/drive/MyDrive/data/초급_프로젝트/dataset/yolo_dataset_v3/labels/train.cache... 1182 images, 0 backgrounds, 0

alumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', nu

WARNING ☒ **val:** Slow image access detected (ping: 134.1±298.5 ms, read: 150.2±202.6 MB/s, size: 1734.1 KB). Use local storage inste

val: Scanning /content/drive/MyDrive/data/초급_프로젝트/dataset/yolo_dataset_v3/labels/val.cache... 307 images, 0 backgrounds, 0 corru

optimizer: AdamW(lr=0.005, momentum=0.937) with parameter groups 77 weight(decay=0.0), 84 weight(decay=0.0005), 83 bias(decay=0.0)

Plotting labels to /content/drive/MyDrive/data/초급_프로젝트/dataset/yolo_runs_v3/yolov8m_oral_drug_v3/labels.jpg...

Image sizes 1280 train, 1280 val

Using 8 dataloader workers

Logging results to /content/drive/MyDrive/data/초급_프로젝트/dataset/yolo_runs_v3/yolov8m_oral_drug_v3

Starting training for 50 epochs...

	Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
WARNING ☒							CUDA out of memory with batch=16. Reducing to batch=8 and retrying (1/3).
train:							Fast image access ☒ (ping: 7.5±10.0 ms, read: 19.8±9.8 MB/s, size: 1802.1 KB)
train:							Scanning /content/drive/MyDrive/data/초급_프로젝트/dataset/yolo_dataset_v3/labels/train.cache... 1182 images, 0 backgrounds, 0
alumentations:							Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', nu
val:							Fast image access ☒ (ping: 18.4±24.0 ms, read: 32.9±26.4 MB/s, size: 1746.0 KB)
val:							Scanning /content/drive/MyDrive/data/초급_프로젝트/dataset/yolo_dataset_v3/labels/val.cache... 307 images, 0 backgrounds, 0 corru
optimizer:							AdamW(lr=0.005, momentum=0.937) with parameter groups 77 weight(decay=0.0), 84 weight(decay=0.0005), 83 bias(decay=0.0)

```
# — 환경 자동 감지 —————
try:
    import google.colab
    ENV = "colab"
except ImportError:
    ENV = "local"

if ENV == "local":
    BASE_DIR = "./dataset"
    RUNS_DIR = "runs/detect"
    FONT_PATH = "C:/Windows/Fonts/malgun.ttf" # 윈도우 맑은 고딕
    # FONT_PATH = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf" # 리눅스/Mac

elif ENV == "colab":
    from google.colab import drive
    drive.mount('/content/drive')
```

```

BASE_DIR = "/content/drive/MyDrive/data/초급_프로젝트/dataset"
RUNS_DIR = "/content/drive/MyDrive/data/초급_프로젝트/runs/detect"

# Colab 한글 폰트 설치
import subprocess
subprocess.run(["apt-get", "install", "-y", "fonts-nanum"], capture_output=True)
FONT_PATH = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf"

# — 한글 폰트 적용 —
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm

fm.fontManager.addfont(FONT_PATH)
font_name = fm.FontProperties(fname=FONT_PATH).get_name()
plt.rcParams["font.family"] = font_name
plt.rcParams["axes.unicode_minus"] = False

print(f" 환경: {ENV}")
print(f" 폰트: {font_name}")
print(f" BASE_DIR: {BASE_DIR}")

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

환경: colab
 폰트: NanumGothic
 BASE_DIR: /content/drive/MyDrive/data/초급_프로젝트/dataset

```

# 세션 복원 셀 (세션 재시작 후 Inference 전 실행)
import os, json
import unicodedata
import numpy as np
from collections import Counter

PROCESSED_JSON_PATH = os.path.join(BASE_DIR, "processed_train_v3.json")
TEST_IMG_DIR = os.path.join(BASE_DIR, "test_images")
YOLO_DATASET_DIR = os.path.join(BASE_DIR, "yolo_dataset_v3")
YOLO_RUNS_DIR = os.path.join(BASE_DIR, "yolo_runs_v3")
SUBMISSION_PATH = os.path.join(BASE_DIR, "20260329_tomas_submission_v04.csv")
YOLO_YAML_PATH = os.path.join(YOLO_DATASET_DIR, "data.yaml")
MODEL_PATH = os.path.join(YOLO_RUNS_DIR, "yolov8m_oral_drug_v3", "weights", "best.pt")

with open(PROCESSED_JSON_PATH, 'r', encoding='utf-8') as f:
    processed_json = json.load(f)

train_ids = processed_json['train_ids']
val_ids = processed_json['val_ids']
remapped_categories = processed_json['categories']

old_id2new = {int(k): v for k, v in processed_json['mappings']['old_id2new'].items()}
new_id2old = {int(k): v for k, v in processed_json['mappings']['new_id2old'].items()}
new_id2name = {int(k): v for k, v in processed_json['mappings']['new_id2name'].items()}

print(f"processed_json 로드 완료")
print(f"train_ids : {len(train_ids)}장 | val_ids : {len(val_ids)}장")
print(f"categories : {len(remapped_categories)}개")
print(f"MODEL_PATH : {'OK' if os.path.exists(MODEL_PATH) else '없음'} ({MODEL_PATH})")

```

processed_json 로드 완료
 train_ids : 1182장 | val_ids : 307장
 categories : 73개
 MODEL_PATH : OK (/content/drive/MyDrive/data/초급_프로젝트/dataset/yolo_runs_v3/yolov8m_oral_drug_v3/weights/best.pt)

```

# Inference + submission.csv 생성
from ultralytics import YOLO

CONF_THRESHOLD = 0.25
IOU_THRESHOLD = 0.5
MAX_DETS = 4

model = YOLO(MODEL_PATH)

test_files = sorted([f for f in os.listdir(TEST_IMG_DIR) if f.endswith('.png')])

rows = []
annotation_id = 1

for f in test_files:
    img_path = os.path.join(TEST_IMG_DIR, f)
    image_id = int(os.path.splitext(f)[0])

    results = model.predict(
        source = img_path,
        imgsiz = 1280,

```

```

        conf      = CONF_THRESHOLD,
        iou       = IOU_THRESHOLD,
        max_det   = MAX_DETS,
        verbose   = False,
    )

    boxes = results[0].boxes
    if boxes is None or len(boxes) == 0:
        continue

    for box in boxes:
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        conf = float(box.conf[0])
        class_id = int(box.cls[0])
        orig_cat = new_id2old[class_id] + 1 # +1 보정

        rows.append({
            'annotation_id': annotation_id,
            'image_id'      : image_id,
            'category_id'   : orig_cat,
            'bbox_x'        : x1,
            'bbox_y'        : y1,
            'bbox_w'        : x2 - x1,
            'bbox_h'        : y2 - y1,
            'score'         : conf,
        })
        annotation_id += 1

    df_submission = pd.DataFrame(rows)
    df_submission.to_csv(SUBMISSION_PATH, index=False)

    print(f"총 탐지 수      : {len(df_submission):,}개")
    print(f"탐지 이미지 수    : {df_submission['image_id'].nunique():,}장 / {len(test_files)}장")
    print(f"이미지당 평균      : {len(df_submission)/df_submission['image_id'].nunique():.2f}개")
    print(f"등장 클래스 수     : {df_submission['category_id'].nunique():,}개")
    print(f"저장 경로         : {SUBMISSION_PATH}")

```

```

총 탐지 수      : 3,230개
탐지 이미지 수   : 843장 / 843장
이미지당 평균   : 3.83개
등장 클래스 수   : 73개
저장 경로       : /content/drive/MyDrive/data/초급_프로젝트/dataset/20260329_tomas_submission_v04.csv

```

6-3. 추론

```

# Inference + submission.csv 생성
from ultralytics import YOLO

CONF_THRESHOLD = 0.25
IOU_THRESHOLD  = 0.5
MAX_DETS       = 4

model = YOLO(MODEL_PATH)

test_files = sorted([f for f in os.listdir(TEST_IMG_DIR) if f.endswith('.png')])

rows = []
annotation_id = 1
score_threshold = 0.5

for f in test_files:
    img_path = os.path.join(TEST_IMG_DIR, f)
    image_id = int(os.path.splitext(f)[0])

    results = model.predict(
        source      = img_path,
        imgsz       = 1280,
        conf        = CONF_THRESHOLD,
        iou         = IOU_THRESHOLD,
        max_det     = MAX_DETS,
        verbose     = False,
    )

    boxes = results[0].boxes
    if boxes is None or len(boxes) == 0:
        continue

    for box in boxes:
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        conf = float(box.conf[0])
        class_id = int(box.cls[0])

```

```
orig_cat = new_id2old[class_id] + 1 # +1 보정
```

```
rows.append({
    'annotation_id': annotation_id,
    'image_id'      : image_id,
    'category_id'   : orig_cat,
    'bbox_x'        : x1,
    'bbox_y'        : y1,
    'bbox_w'        : x2 - x1,
    'bbox_h'        : y2 - y1,
    'score'         : conf,
})
annotation_id += 1
```

```
df_submission = pd.DataFrame(rows)
df_submission.to_csv(SUBMISSION_PATH, index=False)
```

```
print(f"총 탐지 수      : {len(df_submission):,}개")
print(f"탐지 이미지 수  : {df_submission['image_id'].nunique():,}장 / {len(test_files)}장")
print(f"이미지당 평균      : {len(df_submission)/df_submission['image_id'].nunique():.2f}개")
print(f"등장 클래스 수    : {df_submission['category_id'].nunique():,}개")
print(f"저장 경로         : {SUBMISSION_PATH}")
```

```
총 탐지 수      : 3,230개
탐지 이미지 수  : 843장 / 843장
이미지당 평균   : 3.83개
등장 클래스 수  : 73개
저장 경로       : /content/drive/MyDrive/data/초급_프로젝트/dataset/20260329_tomas_submission_v04.csv
```

7. Model 검증

7-1. 학습데이터 확인

- 학습될때 기록된 정보를 바탕으로 학습이 정상적으로 되었는지 확인.

코딩을 시작하거나 AI로 코드를 생성하세요.

코딩을 시작하거나 AI로 코드를 생성하세요.

코딩을 시작하거나 AI로 코드를 생성하세요.

7-2. 시각화 검증

- 학습한 모델을 검증으로 전환하여 실제 test 이미지에서 탐지를 진행.
- 탐지한 사진의 결과를 보고 탐지내용 확인

```
import os
import random
import matplotlib.pyplot as plt
from ultralytics import YOLO

CONF_THRESHOLD = 0.25
IOU_THRESHOLD  = 0.5
MAX_DETS       = 4
NUM_SAMPLES    = 120 # 원하는 개수만 바꾸면 됨
COLS           = 4   # 한 행에 몇 개씩 보여줄지

model = YOLO(MODEL_PATH)

test_files = sorted([f for f in os.listdir(TEST_IMG_DIR) if f.endswith('.png')])

# 랜덤으로 NUM_SAMPLES개 선택
sample_files = random.sample(test_files, min(NUM_SAMPLES, len(test_files)))

# subplot 행/열 계산
rows = (len(sample_files) + COLS - 1) // COLS
fig, axes = plt.subplots(rows, COLS, figsize=(COLS*5, rows*5))

# 2차원 배열 형태의 axes를 1차원으로 변환
axes = axes.flatten()

for idx, f in enumerate(sample_files):
    img_path = os.path.join(TEST_IMG_DIR, f)
    image_id = int(os.path.splitext(f)[0])
```

```

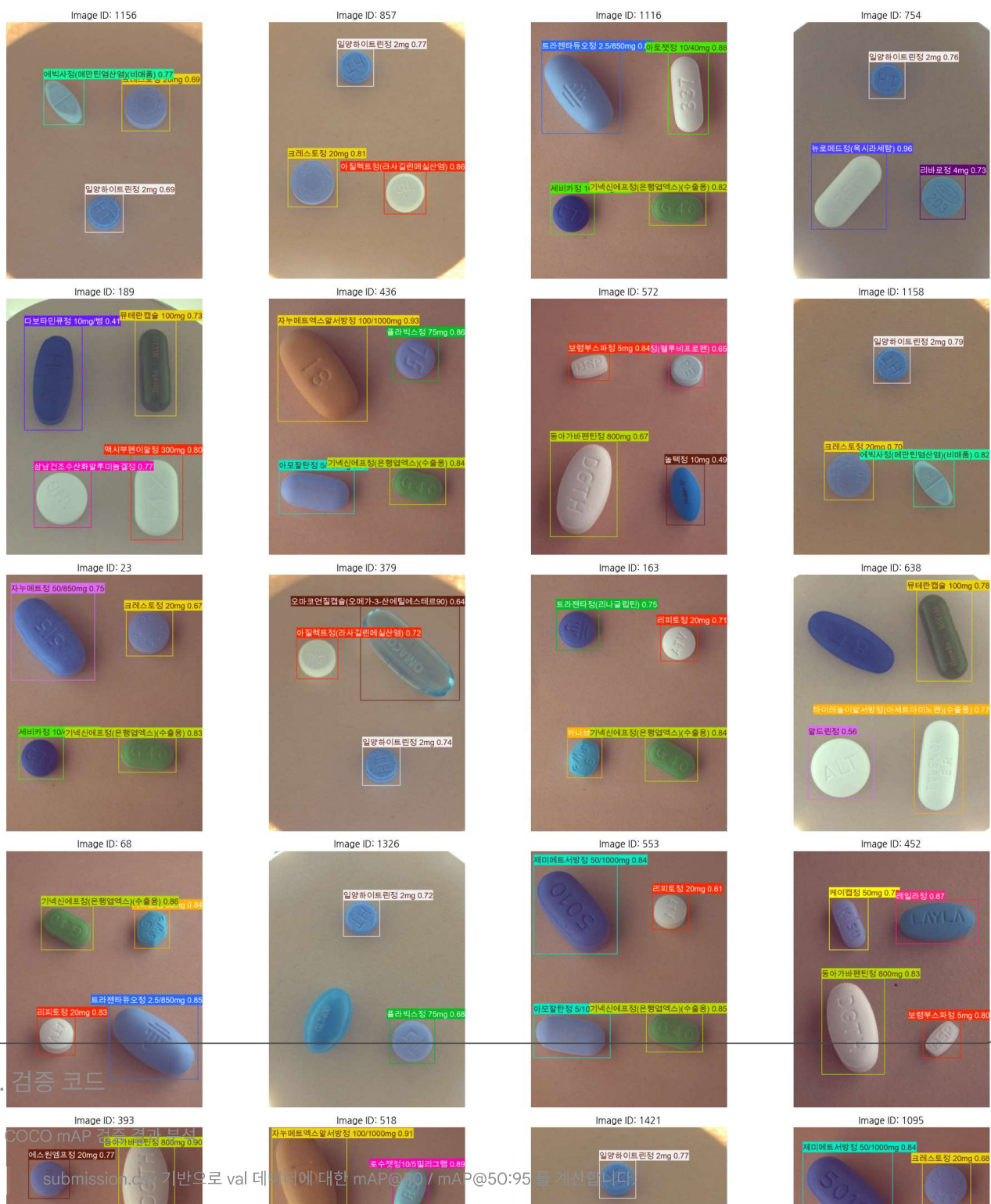
results = model.predict(
    source = img_path,
    imsz = 1280,
    conf = CONF_THRESHOLD,
    iou = IOU_THRESHOLD,
    max_det = MAX_DETS,
    verbose = False,
)

res_img = results[0].plot() # numpy array 반환
axes[idx].imshow(res_img)
axes[idx].axis("off")
axes[idx].set_title(f"Image ID: {image_id}")

# 남은 subplot 칸은 숨기기
for j in range(idx+1, len(axes)):
    axes[j].axis("off")

plt.tight_layout()
plt.show()

```



7-3. 검증 코드

- COCO mAP 검증 코드

submission.csv 기반으로 val 데이터에 대한 mAP@50:95 / mAP@50:95 를 계산합니다

❏ COCO 객체 생성

val GT와 submission에 COCO 형식으로 로드합니다.

- `val_coco_dict` : processed_json에서 val_ids에 해당하는 GT
- `clean_preds` : submission.csv에서 val_ids 필터링 + category_id 역변환한 예측

```
with tempfile.NamedTemporaryFile(mode='w', suffix='.json', delete=False) as tmp:
    json.dump(val_coco_dict, tmp)
    tmp_path = tmp.name
```

```
coco_gt = COCO(tmp_path)
coco_dt = coco_gt.loadRes(clean_preds)
os.unlink(tmp_path)
```

```
print(f" GT 이미지 수 : {len(val_coco_dict['images']):,}장")
print(f" GT 박스 수   : {len(val_coco_dict['annotations']):,}개")
print(f" 예측 박스 수   : {len(clean_preds):,}개")
```

NameError Image ID: 308 Traceback (most recent call last)

```
File ~/tmp/ipykernel_7548/2245966443.py in <module>
----> 1 with tempfile.NamedTemporaryFile(mode='w', suffix='.json', delete=False) as tmp:
      2     json.dump(val_coco_dict, tmp)
      3     tmp_path = tmp.name
      4
      5     coco_gt = COCO(tmp_path)
```

NameError: name 'tempfile' is not defined

❏ 전체 mAP 계산

pycocotools의 공식 COCOval로 전체 mAP를 계산합니다.

mAP@50 IoU 0.5 기준 평균 정밀도
mAP@75 IoU 0.75 기준 (더 엄격)
mAP@50:95 IoU 0.5~0.95 평균 (COCO 공식 지표)

```
coco_eval = COCOeval(coco_gt, coco_dt, "bbox")
coco_eval.evaluate()
coco_eval.accumulate()
coco_eval.summarize()

map50_95 = coco_eval.stats[0]
map50     = coco_eval.stats[1]
map75     = coco_eval.stats[2]

print(f"Wn  mAP@50      : {map50:.4f}  ({map50*100:.2f}%)")
print(f"Wn  mAP@50:95   : {map50_95:.4f} ({map50_95*100:.2f}%)")
print(f"Wn  mAP@75      : {map75:.4f}  ({map75*100:.2f}%)")
```

❏ 클래스별 AP 계산

73개 클래스 각각에 대해 AP@50 / AP@50:95를 계산합니다. 클래스 수가 많아 시간이 다소 걸릴 수 있습니다. (약 1~2분)

```
cat_ids = sorted(coco_gt.getCatIds())
cat_infos = {c['id']: c['name'] for c in coco_gt.loadCats(cat_ids)}
per_class = {}

for i, cat_id in enumerate(cat_ids):
    ce = COCOeval(coco_gt, coco_dt, "bbox")
    ce.params.cat_ids = [cat_id]
    ce.evaluate()
    ce.accumulate()
    ce.summarize()

    ap50_95 = ce.stats[0] if ce.stats[0] != -1 else 0.0
    ap50     = ce.stats[1] if ce.stats[1] != -1 else 0.0
    per_class[cat_infos[cat_id]] = {"ap50": ap50, "ap50_95": ap50_95}
```

```

if (i+1) % 10 == 0:
    print(f" {i+1}/{len(cat_ids)} 완료...")

# AP@50 기준 내림차순 정렬
sorted_by_ap50 = sorted(per_class.items(), key=lambda x: -x[1]['ap50'])
names = [x[0] for x in sorted_by_ap50]
ap50s = [x[1]['ap50'] * 100 for x in sorted_by_ap50]
ap5095s = [x[1]['ap50_95'] * 100 for x in sorted_by_ap50]

detected = sum(1 for v in per_class.values() if v['ap50'] > 0)
print(f"Wn AP@50 > 0 클래스 : {detected} / {len(per_class)}개")
print(f" AP@50 = 0 클래스 : {len(per_class) - detected}개 (미탐지 또는 오탐)")

```

시각화

4개 차트로 결과를 시각화합니다

- 화상: 전체 AP 지표 요약
- 위상: 클래스별 AP@50 분포 히스토그램
- 화하: AP@50 상위 20개 클래스
- 우하: AP@50 하위 20개 클래스 (개선 필요 대상)

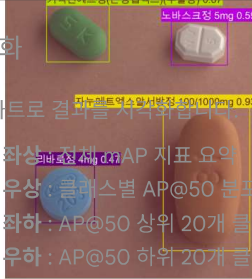


Image ID: 5



Image ID: 468



Image ID: 1170



Image ID: 1493

```

fig, axes = plt.subplots(2, 2, figsize=(20, 18))
fig.suptitle(
    f"COCO mAP 검증 결과 WnAP@50: {map50*100:.2f}% | mAP@50:95: {map50_95*100:.2f}%",
    fontsize=16, fontweight='bold', y=0.98
)

# — (1) 전체 지표 요약 —
ax = axes[0, 0]
metrics = ['mAP@50', 'mAP@75', 'mAP@50:95']
values = [map50*100, map75*100, map50_95*100]
colors = ['#2ecc71', '#3498db', '#e74c3c']
bars = ax.bar(metrics, values, color=colors, width=0.5, edgecolor='white', linewidth=1.5)
for bar, val in zip(bars, values):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
            f'{val:.2f}%', ha='center', va='bottom', fontsize=13, fontweight='bold')
ax.set_ylim(0, max(values) * 1.25 + 5)
ax.set_title('전체 mAP 지표 요약', fontsize=13, fontweight='bold')
ax.set_ylabel('AP (%)')
ax.grid(axis='y', alpha=0.3)
ax.spines[['top', 'right']].set_visible(False)

# — (2) AP@50 분포 히스토그램 —
ax = axes[0, 1]
ap50_vals = np.array(ap50s)
ax.hist(ap50_vals, bins=range(0, 101, 10), color='#3498db', edgecolor='white', linewidth=1.2)
ax.axvline(np.mean(ap50_vals), color='#e74c3c', linestyle='--', linewidth=2,
            label=f'평균 {np.mean(ap50_vals):.1f}%')
ax.set_title('클래스별 AP@50 분포', fontsize=13, fontweight='bold')
ax.set_xlabel('AP@50 (%)')
ax.set_ylabel('클래스 수')
ax.legend(fontsize=11)
ax.grid(axis='y', alpha=0.3)
ax.spines[['top', 'right']].set_visible(False)

# — (3) 상위 20개 클래스 —
ax = axes[1, 0]
top_n = 20
top_names = names[:top_n]
top_ap50 = ap50s[:top_n]
top_ap95 = ap5095s[:top_n]
y_pos = np.arange(top_n)
ax.barh(y_pos, top_ap50, color='#3498db', alpha=0.85, label='AP@50')
ax.barh(y_pos, top_ap95, color='#e74c3c', alpha=0.85, label='AP@50:95')
ax.set_yticks(y_pos)
ax.set_yticklabels(top_names, fontsize=8)
ax.set_xlabel('AP (%)')
ax.set_title(f'AP@50 상위 {top_n}개 클래스', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(axis='x', alpha=0.3)
ax.spines[['top', 'right']].set_visible(False)
ax.invert_yaxis()

# — (4) 하위 20개 클래스 —
ax = axes[1, 1]
bot_names = names[-top_n:][:-1]
bot_ap50 = ap50s[-top_n:][:-1]
bot_ap95 = ap5095s[-top_n:][:-1]

```

자누메트정 50/850mg 0.85